



Lab 05: Logic Coverage

Objective: Learn to identify predicates and clauses in real conditional logic, then apply the semantic logic coverage criteria to them: Predicate Coverage, Clause Coverage, Combinatorial Coverage, the Active Clause Coverage family (GACC, RACC, CACC), and the Inactive Clause Coverage family (GICC, RICC). You will also learn the algebraic method for finding exactly which minor clause values make a clause determine a predicate, and connect infeasibility and subsumption back to what you already found for graphs in Labs 02 through 04.

Introduction

Chapter 8 introduces **logic** as the third structure for modeling software, after graphs. Where graph coverage asked "which paths through the code get executed," logic coverage asks a narrower but complementary question: "which combinations of true/false conditions actually get exercised inside a single decision?" A **predicate** is any expression that evaluates to true or false; it is built out of one or more **clauses** joined by logical operators (AND, OR, NOT, and so on). A clause is a predicate with no logical operators of its own, typically a boolean variable, a relational expression like $x < y$, or a boolean function call.

The lecture uses this notation throughout, and so does this lab: P is the set of all predicates being tested, p is one predicate in P , C is the set of all clauses, C_p is the set of clauses inside predicate p , and c is one clause.

New ideas in this lab:

- **Predicate Coverage (PC) and Clause Coverage (CC):** PC requires each predicate to be tested true and false at least once. CC requires each individual clause to be tested true and false at least once, which is stronger in spirit but, surprisingly, does not guarantee PC.

- **Combinatorial Coverage (CoC):** requires every combination of clause values, 2^N test requirements for N clauses. Exhaustive but expensive.
- **Determination:** a clause c_i determines predicate p if, holding the other (minor) clauses fixed, changing c_i changes the value of p . This is what makes a clause the deciding factor rather than just along for the ride.
- **Active Clause Coverage family:** GACC, RACC, and CACC all require each clause to determine the predicate at least once true and once false, but disagree on whether the minor clauses are allowed to differ between the true case and the false case. This ambiguity is exactly what MC/DC (used by the FAA for safety-critical software) has to resolve.
- **Inactive Clause Coverage family (GICC, RICC):** the opposite idea, deliberately choosing minor clauses so that a clause does *not* determine the predicate, while still covering p true and p false.
- **Infeasibility and subsumption:** just like graphs, some logic test requirements are mathematically impossible to satisfy, and the criteria form a subsumption hierarchy from Predicate Coverage up to Combinatorial Coverage.
- **Making a clause determine a predicate (algebraically):** $p_c = p_{c=\text{true}} \text{ XOR } p_{c=\text{false}}$ gives you a formula for exactly which minor clause values make c determine p , without having to build the whole truth table by hand.

Lab Activity

This is an individual activity. Complete each section on your own and type your answers directly into the boxes below.

Section 1: Predicates and Clauses

Prepare a concise document covering the following.

1. What is the difference between a predicate and a clause? List the logical operators a predicate can be built from, and give one example of each of the three kinds of clause: a boolean variable, a relational expression, and a boolean function call.

2. Translate this sentence into a predicate, then check your own translation for the same kind of mistake the lecture warns about: "A customer gets free shipping if they have a membership and their order is over \$50, or if they are picking up in-store." Write down your clauses (one per condition) and the full predicate using AND, OR, and NOT. Is there any ambiguity in the English that your logic had to resolve one way or another?

3. Using the notation P , p , C , C_p , c , describe in your own words what each symbol refers to, and explain why C_p is written as a function of p rather than being the same set for every predicate.

Section 2: Predicate Coverage and Clause Coverage

The Java method below decides whether a payment can be approved. It has three clauses joined by AND and OR.

```
public static boolean canApprovePayment(  
    int amount,  
    int limit,  
    boolean isVerified,  
    int riskScore,  
    int threshold) {  
  
    return (amount < limit && isVerified)  
        || (riskScore >= threshold);  
}
```

Task A: Identify the clauses

Write down the predicate p in terms of three clauses c_1 , c_2 , c_3 (name them yourself), and state what each clause corresponds to in the code.

Task B: Predicate Coverage

Write the TR for Predicate Coverage on p (two requirements: p true, p false). Give one test case (concrete values for amount, limit, isVerified, riskScore, threshold) for each requirement, and show the substitution that proves p evaluates the way you claim.

Task C: Clause Coverage

Write the TR for Clause Coverage (each of c_1 , c_2 , c_3 evaluates true and false somewhere in your test set). Try to do it with only two test cases, the same way the lecture's own Clause Coverage example uses exactly two tests. Show which test makes each clause true and which makes it false.

Task D: Does Clause Coverage guarantee Predicate Coverage?

Find two test cases for this same predicate that together satisfy 100% Clause Coverage (all three clauses take both true and false across the set) but do NOT satisfy Predicate Coverage (the predicate takes only one of its two values across the whole set). State the two test cases and the predicate's value for each. Then explain in your own words why this gap between CC and PC exists, and what a "Combinatorial" style criterion would need to add to close it.

Section 3: Combinatorial Coverage

Combinatorial Coverage (CoC) asks for every possible combination of clause truth values, not just enough to cover each clause individually.

Task E: Build the full truth table

Using the same predicate from Section 2 ($p = (c1 \text{ AND } c2) \text{ OR } c3$), build the complete truth table: all $2^3 = 8$ rows of $c1$, $c2$, $c3$, and the resulting value of p for each row. This is the full TR for Combinatorial Coverage on p .

#	c1	c2	c3	p
1	T/F	T/F	T/F	T/F
2	T/F	T/F	T/F	T/F
3	T/F	T/F	T/F	T/F
4	T/F	T/F	T/F	T/F
5	T/F	T/F	T/F	T/F
6	T/F	T/F	T/F	T/F
7	T/F	T/F	T/F	T/F
8	T/F	T/F	T/F	T/F

The lecture reports that from a study of over 400,000 real predicates, 88.5% had only one clause, and only 0.65% had four clauses or more. Given that CoC costs 2^N test cases, what does that statistic imply about how often CoC is actually practical to use, and how often Predicate Coverage alone is already enough?

This predicate has 3 clauses, so CoC costs 8 tests. If you added a 4th clause to a predicate like this with another AND, how many test cases would CoC require, and is that number still practical for a developer to write by hand?

Section 4: Active Clauses and the ACC Family

This section uses a new example, a loan approval check with the same "AND of an OR" shape the lecture itself uses to introduce determination.

```
public static boolean isLoanApproved(  
    boolean hasStableIncome,  
    boolean hasGoodCredit,  
    boolean hasCollateral) {  
  
    return hasStableIncome && (hasGoodCredit || hasCollateral);  
}
```

Recall determination: clause c_i (the major clause) determines predicate p if, for some fixed values of the other (minor) clauses, flipping c_i from true to false (or back) flips the value of p . Call the three clauses here a (`hasStableIncome`), b (`hasGoodCredit`), c (`hasCollateral`), so $p = a \text{ AND } (b \text{ OR } c)$.

Task F: Find the determining row-pairs

Build the 8-row truth table for $p = a \text{ AND } (b \text{ OR } c)$ (you can reuse the table shape from Task E). For each of a , b , and c in turn, find every pair of rows where that clause is the **ONLY** one that changes and the value of p also changes, that is, every pair that shows that clause determining p . You should find noticeably more determining pairs for one clause than for the other two, explain why that asymmetry makes sense given the AND/OR structure of p .

Task G: GACC vs. RACC vs. CACC

Focus on major clause a . Using your Task F results, write down: (1) a pair of test cases that satisfies RACC for a (the minor clauses b and c must be identical when a is true and when a is false), and (2) a pair of test cases that satisfies CACC for a but would NOT satisfy RACC (the minor clauses are allowed to differ, as long as p still comes out true for one and false for the other). In your own words, explain the ambiguity the lecture describes, "do the minor clauses have to have the same values when the major clause is true and false," and why RACC's answer to that question (yes) is stricter than CACC's (no).

Section 5: Inactive Clause Coverage

Inactive Clause Coverage flips the goal: deliberately pick minor clause values so that the major clause does **not** determine the predicate, while still requiring all four combinations: ci true with p true, ci false with p true, ci true with p false, ci false with p false.

Task H: Attempt ICC on two different clauses

Still using $p = a \text{ AND } (b \text{ OR } c)$ from Section 4. First, attempt to satisfy all four ICC requirements for major clause a (choosing b and c so that a does not determine p). You should find that one of the four requirements cannot be satisfied for ANY choice of b and c, identify which one and explain why it is fundamentally infeasible given that $p = a \text{ AND } (...)$. Then attempt ICC for major clause b instead, and show that all four requirements are satisfiable there (they do not need to use the same minor values for all four, only within each same-p-value pair).

Section 6: Infeasibility and Subsumption

Consider this predicate, built from three relational clauses over real numbers x, y, and z:

$$(x > y \ \&\& \ y > z) \ \&\& \ z > x$$

Task I: Explain the infeasibility

Show that there is no possible assignment of real numbers to x, y, and z that makes all three clauses true at once (you can reason about it directly, or pick a few numeric examples to build intuition first). Then connect this back to Lab 03's Task I: classifyAge had an infeasible NODE because of a contradiction between two conditions on the same variable. What kind of contradiction makes THIS predicate infeasible instead, and is it the same kind or a different kind?

Logic Criteria Subsumption (weakest to strongest): Predicate Coverage → Clause Coverage → General Active/Inactive Clause Coverage (GACC/GICC) → Correlated Active Clause Coverage (CACC) → Restricted Active/Inactive Clause Coverage (RACC/RICC) → Combinatorial Coverage. Satisfying a stronger criterion guarantees satisfying every weaker one below it.

Task J: Subsumption in practice

RACC often produces infeasible test requirements that CACC does not (this is exactly the pattern you may have already run into in Task H, or would if you tried ICC's restricted form). Given that RACC still sits above CACC in the subsumption hierarchy (RACC is "stronger"), what does it mean for a stronger criterion to be less practical than a weaker one? Is subsumption alone ever a good enough reason to pick one criterion over another for a real project?

Section 7: Making a Clause Determine a Predicate

Building a full truth table by hand gets impractical as predicates grow. The algebraic shortcut: $p_c = p_{c=\text{true}} \text{ XOR } p_{c=\text{false}}$ where $p_{c=\text{true}}$ means "p with every occurrence of c replaced by true." Solving p_c tells you exactly which values of the OTHER clauses make c determine p.

XOR identity reminders: A XOR B means "A or B is true, but not both." Two useful shortcuts: A XOR (A AND b) simplifies to A AND (NOT b). A XOR (A OR b) simplifies to (NOT A) AND b.

Task K: Two warm-up derivations

By hand, derive p_a and p_b for $p = a \text{ AND } b$, and separately for $p = a \text{ OR } b$ (four small derivations total). Show your substitution of true/false for each occurrence of the clause you are solving for, and simplify using the XOR identities above.

Task L: Apply it to isLoanApproved

Now derive p_a for $p = a \text{ AND } (b \text{ OR } c)$ (Section 4's predicate) algebraically. Simplify your answer down to a short boolean expression in b and c . Then check it against your Task F answer: does the condition you derived algebraically actually match every determining row-pair you found by hand for major clause a ?

Discussion

Active Clause Coverage is described in the lecture as "a form of MC/DC, which is required by the FAA for safety-critical software." Based on what you now know about the difference between RACC and CACC, why might regulators specifically care about which variant of MC/DC a test suite actually implements, rather than just saying "use MC/DC"?

Graphs (Labs 02 to 04) and logic (this lab) are two different "structures" for modeling the same underlying code, according to the lecture's own framing. Give one example of a fault that a logic criterion (PC, CC, CoC, or an ACC/ICC variant) would catch but that Edge-Pair or Prime Path Coverage could miss entirely, even at 100%.

